

[НАЗВАНИЕ УЧЕБНОГО ЗАВЕДЕНИЯ]

[КАФЕДРА / ОТДЕЛЕНИЕ]

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по дисциплине «Программирование на языках высокого уровня»

**на тему «Разработка программного средства обработки изображений
ImageToolkit»**

Выполнил: [ФИО СТУДЕНТА]

Группа: [ГРУППА]

Проверил: [ФИО РУКОВОДИТЕЛЯ]

[ГОРОД] 2026

Содержание

1	Введение	3
2	Постановка задачи на курсовую работу	5
3	Теоретические сведения	8
3.1	Растровое изображение как объект программной обработки.....	8
3.2	Цветовые каналы и поточечные преобразования	8
3.3	Геометрические преобразования изображения.....	9
3.4	Форматы графических файлов и ввод-вывод данных	10
3.5	Средства разработки настольных приложений на С# и .NET	11
3.6	Архитектурные принципы построения программного средства	11
3.7	Мультимодальные модели и распознавание объектов на фотографии	12
4	Алгоритмизация.....	15
4.1	Разработанные структуры данных проекта	15
4.2	Общая схема локальной обработки изображения	17
4.3	Алгоритмы поточечных преобразований	18
4.4	Алгоритмы геометрических преобразований	19
4.5	Алгоритм распознавания объектов на фотографии.....	20
4.6	Алгоритмы пользовательского интерфейса и сохранения результатов	22
5	Программное средство	24
5.1	Состав и организация решения	24
5.2	Описание пользовательского интерфейса	25
5.3	Поддерживаемые функции программы	25
5.4	Порядок работы пользователя с программой	26
5.5	Иллюстрации интерфейса программы	27
5.6	Эксплуатационные особенности и ограничения.....	28
6	Заключение	30
7	Список использованной литературы	31

1 Введение

Современные цифровые изображения выступают одним из наиболее распространенных видов прикладных данных. Они применяются в образовательных системах, в документообороте, в рекламных и инженерных материалах, в социальных сетях, в медиаархивах и в большом количестве локальных пользовательских сценариев. По этой причине задачи загрузки, хранения, визуализации и преобразования растровых изображений остаются актуальными как в профессиональной разработке, так и в учебной практике.

Курсовая работа по программированию должна не только демонстрировать знание синтаксиса языка, но и подтверждать умение строить завершенное программное средство с графическим интерфейсом, внутренней моделью данных, набором алгоритмов, обработкой ошибок и оформленной программной документацией. Именно поэтому тема разработки настольного приложения для обработки изображений представляется удачной: она объединяет задачи алгоритмизации, проектирования интерфейса, модульного разбиения системы и организации взаимодействия пользователя с программой.

Дополнительную актуальность выбранной теме придает быстрое развитие интеллектуальных функций обработки визуальной информации. Если ранее учебные приложения ограничивались только локальными фильтрами, то в настоящее время даже компактное настольное средство может быть дополнено возможностью анализа содержимого изображения. Это позволяет расширить практическую ценность проекта и показать интеграцию классической локальной обработки пиксельных данных с современными облачными сервисами распознавания объектов.

Целью курсовой работы является закрепление теоретических знаний и формирование практических навыков разработки прикладного программного средства на языке C# с графическим интерфейсом Windows, реализующего обработку растровых изображений и дополнительный интеллектуальный анализ

загруженной фотографии. В рамках работы необходимо пройти все этапы создания продукта: от анализа предметной области и выбора архитектуры до реализации

и подготовки пояснительной записки.

Объектом исследования выступают методы программной обработки и интерпретации растровых изображений в настольных приложениях. Предмет исследования составляют алгоритмы поточечного и геометрического преобразования изображения, способы представления графических данных во внутренней памяти, а также архитектурные решения, обеспечивающие совместную работу локальных функций обработки и удаленного сервиса распознавания объектов.

Практическая значимость работы заключается в создании законченного прикладного инструмента ImageToolkit. Разработанная программа позволяет пользователю открывать графические файлы, выполнять над ними базовые преобразования, получать визуальный предпросмотр, сохранять итоговый результат и формировать отдельный отчет со списком распознанных на изображении объектов, отсортированных по имени. Такой набор функций делает проект не только учебным, но и пригодным для демонстраций, лабораторных занятий и первичного знакомства с визуальными данными.

Логика изложения материала в пояснительной записке выстроена от общих теоретических оснований к конкретной реализации. Сначала формулируется постановка задачи, затем рассматриваются теоретические сведения по растровой графике, средствам разработки и компьютерному зрению, после чего приводятся алгоритмические решения, описание программного средства, результаты проверки его работоспособности и выводы по выполненной работе.

2 Постановка задачи на курсовую работу

В рамках курсовой работы требуется разработать настольное программное средство ImageToolkit, предназначенное для работы с растровыми изображениями в среде Windows. Пользователь должен иметь возможность загрузить изображение из локального файла, увидеть его исходный вид, выбрать преобразование, оценить предпросмотр результата и сохранить обработанный графический файл в один из поддерживаемых форматов.

Ключевым содержанием задачи является реализация собственных алгоритмов обработки изображения. Программа не должна сводиться только к использованию готового графического редактора или сторонней библиотеки высокого уровня. Внутри проекта необходимо организовать собственную модель хранения пиксельных данных, каталог операций и вычислительные методы, обеспечивающие перевод в оттенки серого, инверсию, коррекцию яркости, пороговую бинаризацию, зеркальное отражение и поворот изображения.

Дополнительно в проект вводится интеллектуальная функция распознавания объектов на фотографии. Для ее реализации требуется интеграция с внешним API мультимодальной модели, способной принимать изображение и возвращать структурированный результат анализа. По условию задачи информация о распознанных объектах должна сохраняться в отдельный файл. Для повышения удобства последующей обработки список объектов необходимо приводить к нормализованному виду и сортировать по имени.

При проектировании следует обеспечить наглядный и эргономичный пользовательский интерфейс. Он должен содержать кнопки открытия файла, применения операции, сброса состояния, сохранения результата и запуска распознавания объектов, а также зону выбора операции, область настройки параметра и текстовый блок со служебной информацией. Визуальное представление программы должно давать пользователю возможность видеть

текущее изображение, результат локального преобразования и текстовый отчет распознавания без перегрузки окна лишними областями сравнения.

Для достижения цели разработки в работе последовательно решаются следующие задачи:

1. проанализировать предметную область программной обработки растровых изображений и определить набор функций, достаточный для учебного программного средства;

2. обосновать выбор языка C#, платформы .NET и технологии WPF как основы для построения настольного графического интерфейса;

3. спроектировать внутренние структуры данных для хранения изображения, цветов пикселей, параметров операций и результатов распознавания объектов;

4. разработать алгоритмы локальной обработки изображений, обеспечивающие поточечные и геометрические преобразования;

5. реализовать пользовательский интерфейс, поддерживающий загрузку файлов, предпросмотр, применение операций и сохранение результатов;

6. интегрировать программное средство с API мультимодальной модели для распознавания объектов на фотографии и извлечения краткого описания сцены;

7. организовать сохранение результатов распознавания в текстовый или JSON-файл с сортировкой объектов по имени;

Таким образом, постановка задачи охватывает как вычислительную составляющую, так и вопросы проектирования интерфейса, интеграции внешнего сервиса, контроля качества и документирования разработки.

Таблица 2.1 - Соответствие пользовательских потребностей и функций разрабатываемой программы

Потребность пользователя	Функция системы	Ожидаемый результат
--------------------------	-----------------	---------------------

Быстро открыть изображение и проверить его содержимое	Загрузка файла и отображение исходного изображения	Изображение появляется в области просмотра без ручной конвертации пользователем
Оценить действие фильтра до окончательного применения	Формирование предпросмотра на рабочей копии	Пользователь видит измененный вариант без потери исходных данных
Выполнить типовые преобразования растровых данных	Набор локальных операций обработки	Получение визуально корректного результата для базовых сценариев
Понять, какие объекты присутствуют на фотографии	Интеграция с сервисом распознавания объектов	Сформирован упорядоченный список объектов и краткое описание сцены
Сохранить итоги работы для последующего использования	Экспорт изображения и отчета о распознавании	Создан графический файл и отдельный текстовый либо JSON-документ

3 Теоретические сведения

3.1 Растровое изображение как объект программной обработки

Растровое изображение представляет собой упорядоченный набор пикселей, размещенных в прямоугольной двумерной сетке. Каждый пиксель хранит информацию о цвете, а в ряде форматов дополнительно и об уровне прозрачности. С точки зрения программной реализации изображение удобно рассматривать как линейный массив байтов, где соседние элементы описывают каналы одного пикселя, а переход между строками осуществляется с учетом ширины изображения и числа байтов на точку.

Такой способ представления особенно полезен при реализации собственных алгоритмов обработки. Разработчик получает возможность последовательно обходить все пиксели изображения и вычислять новые значения каналов без обращения к сложным графическим библиотекам высокого уровня. В учебной практике это важно, поскольку позволяет увидеть прямую связь между математической формулой преобразования и реальным визуальным результатом.

Ключевыми характеристиками растрового изображения выступают ширина, высота, цветовая глубина и порядок расположения каналов. Для настольных приложений Windows распространенным является формат BGRA32, в котором на каждый пиксель отводится четыре байта: синий, зеленый, красный и альфа-канал. Выбор именно такого формата упрощает совместную работу пользовательского интерфейса WPF и внутренней вычислительной модели, так как исключает неоднозначность при конвертации входных данных.

3.2 Цветовые каналы и поточечные преобразования

Значительная часть операций обработки изображений относится к поточечным, то есть вычисляет новое значение пикселя, опираясь только на исходные данные этого же пикселя. К таким преобразованиям относятся инверсия, изменение яркости, перевод в оттенки серого и бинаризация по порогу.

Их достоинством является простота реализации и линейная вычислительная сложность относительно числа пикселей изображения.

Для корректной работы с цветом важно учитывать различный вклад каналов в воспринимаемую яркость. Человеческое зрение по-разному реагирует на красную, зеленую и синюю составляющие, поэтому при переводе в оттенки серого обычно используется взвешенная формула яркости, в которой зеленый канал оказывает наибольшее влияние. Такой подход обеспечивает более естественный визуальный результат по сравнению с простым средним арифметическим.

Операция инверсии строится на замене каждого цветового значения на величину, равную разности между максимально возможным значением канала и исходным значением. Изменение яркости реализуется путем добавления или вычитания некоторой дельты ко всем RGB-каналам с обязательным ограничением диапазона. Пороговая обработка переводит изображение в двоичный вид и потому особенно наглядна при демонстрации сравнений, условных переходов и работы с диапазоном 0..255.

3.3 Геометрические преобразования изображения

Геометрические операции отличаются от поточечных тем, что изменяют пространственное расположение пикселей. В этом случае необходимо пересчитывать координаты целевых элементов и переносить данные в новый буфер изображения. Даже относительно простые действия, такие как зеркальное отражение и поворот на девяносто градусов, наглядно показывают необходимость правильной адресации элементов массива и аккуратного обращения с размерами кадра.

Зеркальное отражение по горизонтали можно трактовать как перестановку пикселей относительно вертикальной оси изображения. Для каждого элемента исходной строки определяется новая позиция, симметричная текущей. Эта

операция не изменяет размеры кадра, но полностью меняет пространственное соответствие между исходными и результирующими координатами.

Поворот вправо требует уже не только пересчета координат, но и перестановки размерностей. Новая ширина становится равной исходной высоте, а новая высота - исходной ширине. Реализация подобного преобразования хорошо демонстрирует отличие между логикой индексирования двумерного массива и его физическим хранением в виде линейной последовательности байтов в памяти.

3.4 Форматы графических файлов и ввод-вывод данных

Для практического использования программное средство должно работать с распространенными графическими форматами. На стадии ввода особенно важна гибкость, так как пользователь может иметь изображения в форматах PNG, JPEG, BMP, GIF, TIFF или WebP. На стадии сохранения целесообразно ограничиться наиболее массовыми вариантами, способными покрыть учебные сценарии без усложнения интерфейса и экспортных алгоритмов.

При открытии файла приложение сталкивается не только с необходимостью декодировать графические данные, но и с задачей привести их к единому внутреннему представлению. Даже если входное изображение хранится в другом пиксельном формате, обработка будет существенно проще, если на следующем этапе все данные преобразуются в BGRA32. Единая модель хранения уменьшает число ветвлений в вычислительном коде и делает поведение системы более предсказуемым.

Сохранение результата тесно связано с кодерами изображения. Выбор кодера определяется расширением выходного файла, а дополнительные параметры, например качество JPEG, влияют на компромисс между размером и точностью сохранения. Наличие стандартных диалогов открытия и сохранения делает приложение понятным пользователю, а также снижает риск ошибок при работе с путями и форматами файлов.

3.5 Средства разработки настольных приложений на C# и .NET

Платформа .NET предоставляет развитый набор средств для построения настольных приложений, библиотек классов и тестовых проектов. Язык C# сочетает строгость статической типизации, высокую выразительность синтаксиса и хорошую интеграцию с инструментами Visual Studio и командной строкой dotnet. Это делает его удобным выбором для учебной курсовой работы, где важно одновременно продемонстрировать и алгоритмическое мышление, и инженерную дисциплину.

Технология Windows Presentation Foundation ориентирована на декларативное описание интерфейса в XAML и событийную модель взаимодействия в коде C#. Для приложений, подобных ImageToolkit, WPF удобен наличием стандартных контейнеров, средств работы с изображениями, возможностью гибкой компоновки элементов и естественной поддержкой адаптации интерфейса под разные размеры окна. При этом часть логики пользовательского интерфейса может оставаться прозрачной и читаемой даже в рамках учебного проекта без сложной инфраструктуры.

Важным преимуществом .NET является естественная поддержка модульного разбиения решения. Отдельная библиотека классов позволяет хранить вычислительные алгоритмы независимо от пользовательского интерфейса, а модульная организация кода упрощает сопровождение и поэтапное развитие приложения. Такое разбиение повышает сопровождаемость системы и делает архитектуру проекта ближе к промышленным практикам разработки.

3.6 Архитектурные принципы построения программного средства

Даже для относительно компактного учебного приложения важно соблюдать принцип разделения ответственности. Вычислительная логика обработки изображений не должна зависеть от элементов графического

интерфейса, поскольку это усложняет сопровождение, ухудшает повторное использование кода и затрудняет последующее расширение функциональности. По этой причине в проекте выделено ядро обработки изображений, представленное библиотекой ImageToolkit.Core.

В пользовательском проекте размещается логика взаимодействия с файловой системой, визуальными элементами окна и внешним сервисом распознавания объектов. Такой подход позволяет оставить вычислительное ядро чистым и предсказуемым, а также упрощает сопровождение кода: изменения интерфейса не требуют переписывания пиксельных алгоритмов, а новые операции обработки можно добавлять без полной переработки окна приложения.

Архитектурно система имеет гибридный характер. Базовые преобразования изображения выполняются локально и мгновенно, тогда как функция интеллектуального анализа опирается на удаленный API. Теоретически подобная схема сочетает сильные стороны обоих подходов: скорость и автономность классической обработки с одной стороны и расширенные аналитические возможности мультимодальной модели с другой.

3.7 Мультимодальные модели и распознавание объектов на фотографии

Современные мультимодальные модели способны воспринимать не только текст, но и визуальные входные данные. При передаче изображения такая модель может описать сцену, выделить объекты, прочесть видимый текст и вернуть результат в структурированном виде. В отличие от классических детекторов компьютерного зрения, ориентированных на точные координаты и рамки, мультимодальная модель особенно удобна в пользовательских задачах, где требуется качественное описание содержимого фотографии.

Для настольного учебного приложения разумно использовать именно такой тип анализа, поскольку он позволяет решить сразу несколько задач: определить основные объекты на снимке, получить краткое описание сцены и,

при необходимости, извлечь текст с изображения. При этом пользователь получает результат в привычной текстовой форме, а сама программа может сохранить его в текстовый либо JSON-файл без разработки собственных алгоритмов машинного зрения.

Однако применение внешнего сервиса накладывает и ограничения. Требуется подключение к сети, наличие действующего API-ключа, учет времени ожидания ответа и корректная обработка возможных ошибок. С теоретической точки зрения это означает, что интеллектуальная функция должна быть оформлена как самостоятельный сервисный модуль, изолированный от локальной логики обработки, чтобы сбой сети не нарушал работу базовых функций приложения.

Таблица 3.1 - Обоснование выбора технологий для проекта ImageToolkit

Технология	Роль в проекте	Причина выбора
C#	Основной язык реализации	Статическая типизация, развитые средства ООП, интеграция с .NET и Visual Studio
.NET	Платформа исполнения	Поддержка настольных приложений, библиотек классов и модульного тестирования
WPF	Построение графического интерфейса	Гибкая XAML-разметка, контейнеры компоновки, встроенная работа с изображениями
OpenAI API	Распознавание объектов на изображении	Поддержка мультимодального ввода, структурированного JSON-ответа и OCR-сценариев

Таблица 3.2 - Сравнение локальной обработки изображений и внешнего интеллектуального анализа

Критерий	Локальные алгоритмы	Облачный сервис распознавания
Источник вычислений	Компьютер пользователя	Удаленный API
Скорость отклика	Мгновенная для небольших изображений	Зависит от сети и времени ответа модели
Тип результата	Новый набор пикселей	Текстовое описание и список объектов
Необходимость сети	Не требуется	Обязательна
Основная область применения	Фильтры и преобразования	Смысловой анализ сцены и OCR

4 Алгоритмизация

Алгоритмизация в данном проекте охватывает как структуры хранения изображения, так и функции, управляющие жизненным циклом данных в приложении. Для корректной реализации требовалось определить минимальный, но достаточный набор сущностей, который описывает исходный кадр, рабочую копию, параметры выбранной операции и результаты интеллектуального анализа.

При разработке алгоритмов особое внимание уделялось двум требованиям. Во-первых, все вычисления должны быть понятны с точки зрения учебной демонстрации и не скрывать логику преобразования за внешними библиотеками высокого уровня. Во-вторых, алгоритмы обязаны оставаться изолированными и предсказуемыми, поэтому каждая операция проектировалась как чистая функция, возвращающая новый объект изображения и не изменяющая исходные данные неявным образом.

4.1 Разработанные структуры данных проекта

Центральной структурой данных проекта является объект `ImageFrame`. Он инкапсулирует ширину и высоту изображения, а также линейный массив байтов в формате BGRA32. Эта структура выступает внутренним представлением кадра и используется всеми локальными алгоритмами. Благодаря такому решению операции работают поверх одного и того же типа данных независимо от того, было изображение загружено из PNG, JPEG или другого поддерживаемого файла.

Для доступа к отдельному пикселю введена компактная структура `PixelColor`, хранящая четыре канала цвета. Выделение этой структуры упрощает локальную работу с данными и позволяет сравнивать ожидаемые и фактические значения цвета в понятной форме. Параллельно с ней применяется перечисление `ImageOperationType`, фиксирующее набор доступных операций, и дескриптор

ImageOperationDescriptor, который связывает внутренний тип операции с отображаемым названием и диапазоном параметров.

После добавления интеллектуальной функции в проект была введена и отдельная модель результата распознавания. Она включает сущности RecognizedObject и ImageRecognitionResult, позволяющие хранить краткое описание сцены, распознанный текст и список объектов. Важно, что на уровне модели уже предусмотрена нормализация: удаление пустых названий, объединение дублей, суммирование количества одинаковых объектов и сортировка по имени.

Таблица 4.1 - Основные структуры данных и их назначение

Структура	Назначение	Ключевые поля или свойства
ImageFrame	Внутреннее представление изображения	Width, Height, массив байтов BGRA32
PixelColor	Чтение и сравнение отдельного пикселя	R, G, B, A
ImageOperationRequest	Параметры вызова локальной операции	Тип операции и целочисленный параметр
ImageOperationDescriptor	Описание операции для интерфейса	Название, описание, диапазон параметра
RecognizedObject	Один найденный объект	Имя, количество, краткое описание
ImageRecognitionResult	Итог анализа фотографии	Описание сцены, список объектов, извлеченный текст

Листинг 4.1 - Псевдокод работы со структурой кадра изображения

1. получить из графического файла ширину, высоту и буфер пикселей;
2. проверить, что ширина и высота положительны, а длина буфера соответствует формуле $width * height * 4$;

3. сохранить метаданные кадра и копию буфера внутри объекта `ImageFrame`;

4. при необходимости предоставить методы копирования буфера и чтения отдельного пикселя по координатам.

4.2 Общая схема локальной обработки изображения

После открытия файла изображение приводится к формату BGRA32 и помещается в объект исходного кадра. На его основе создается рабочая копия, с которой пользователь выполняет дальнейшие операции. Такое разделение позволяет в любой момент вернуться к исходному состоянию без повторного чтения файла и без накопления ошибок из-за повторного декодирования.

Важным элементом общей схемы выступает предпросмотр. При каждом изменении выбранной операции или ее параметра программа не изменяет рабочую копию напрямую, а формирует отдельный временный кадр. Пользователь визуально оценивает его и только после нажатия кнопки применения переносит результат в рабочее состояние. С алгоритмической точки зрения это означает наличие трех логических состояний: исходный кадр, текущая рабочая копия и временный предпросмотр.

Таблица 4.2 - Представление метода `Apply` как черного ящика

Входные данные	Имя функции	Результат
Объект <code>ImageFrame</code> и запрос <code>ImageOperationRequest</code>	<code>ImageProcessor.Apply</code>	Новый объект <code>ImageFrame</code> , полученный после выполнения выбранного преобразования

Листинг 4.2 - Псевдокод общей схемы применения локальной операции

1. получить текущую рабочую копию изображения;

2. определить выбранный тип операции и значение параметра, если он поддерживается;
3. передать кадр и запрос в метод Apply;
4. получить новый кадр и отобразить его как предпросмотр;
5. при подтверждении пользователем заменить рабочую копию на результат предпросмотра.

4.3 Алгоритмы поточечных преобразований

Поточечные алгоритмы обладают общей вычислительной схемой: они последовательно обходят буфер пикселей, читают значения каналов текущего элемента, вычисляют новые значения и записывают их в результирующий буфер. Такой подход удобен тем, что не требует сложных зависимостей между соседними пикселями и хорошо демонстрирует роль арифметических операций, округления и ограничений диапазона.

Для перевода в оттенки серого используется формула взвешенной яркости, которая учитывает различный вклад каждого канала в итоговое восприятие изображения. Инверсия реализуется особенно наглядно, поскольку для каждого канала вычисляется значение 255 минус исходное. Изменение яркости отличается от этих операций необходимостью применять ограничение диапазона 0..255, а пороговая обработка дополняет схему сравнением вычисленной яркости с заданным пользователем порогом.

Таблица 4.3 - Представление алгоритма перевода в оттенки серого как черного ящика

Входные данные	Имя функции	Результат
Исходный кадр в формате BGRA32	ApplyGrayscale	Новый кадр, где для каждого пикселя установлена одна и та же яркость по трем цветовым каналам

Листинг 4.3 - Псевдокод поточечной обработки пикселя

1. для каждого пикселя прочитать значения синего, зеленого, красного и альфа-канала;
2. если выбрана операция Grayscale, вычислить яркость по формуле $0.114 * B + 0.587 * G + 0.299 * R$;
3. если выбрана операция Invert, заменить каждый RGB-канал на 255 минус текущее значение;
4. если выбрана операция Brightness, прибавить смещение ко всем RGB-каналам и ограничить результат диапазоном 0..255;
5. если выбрана операция Threshold, вычислить яркость и сравнить ее с порогом, после чего записать либо 0, либо 255;
6. альфа-канал перенести в результат без изменения.

4.4 Алгоритмы геометрических преобразований

Геометрические преобразования требуют уже не пересчета значения цвета как такового, а поиска новой позиции пикселя в целевом буфере. Для этого необходимо явно работать с координатами x и y , вычисляя смещения в линейном массиве. В отличие от поточечных операций здесь особенно важно не перепутать источник и приемник данных, иначе результатом станет частичное наложение старых и новых значений.

При горизонтальном отражении размеры изображения остаются неизменными. Для каждой строки исходного кадра пиксель с координатой x переносится на позицию $width - 1 - x$. При повороте вправо схема усложняется: для нового кадра заранее вычисляются размеры, после чего для каждой точки определяется пара новых координат. Фактически алгоритм представляет собой частный случай отображения двумерной матрицы с перестановкой осей.

Таблица 4.4 - Представление алгоритма поворота изображения как черного ящика

Входные данные	Имя функции	Результат
Исходный кадр шириной W и высотой H	ApplyRotateRight	Новый кадр шириной H и высотой W с пикселями, переставленными по правилу поворота на 90 градусов

Листинг 4.4 - Псевдокод геометрических преобразований

1. создать результирующий буфер требуемого размера;
2. для каждой координаты исходного изображения определить смещение исходного пикселя в массиве;
3. если выбрано зеркальное отражение, вычислить новую координату как $width - 1 - x$ и записать пиксель в тот же ряд;
4. если выбрано вращение, определить $targetX$ как $height - 1 - y$, а $targetY$ как x ;
5. перенести четыре байта пикселя в вычисленное место результирующего буфера;
6. после завершения обхода вернуть новый объект `ImageFrame`.

4.5 Алгоритм распознавания объектов на фотографии

Логика интеллектуального анализа начинается с выбора актуального кадра. Пользователь может распознавать либо только что загруженное изображение, либо уже обработанный результат, находящийся в области предпросмотра. Для передачи во внешний сервис кадр сначала кодируется в PNG, поскольку этот формат стабильно поддерживается, не приводит к

дополнительным потерям качества и легко помещается в текстовую форму после кодирования Base64.

Сетевой сервис формирует запрос к мультимодальной модели, в котором одновременно передается текстовая инструкция и само изображение. Важнейшим алгоритмическим элементом здесь выступает принудительное требование структурированного JSON-ответа. За счет этого программа получает не произвольный текст, а строго заданную структуру с описанием сцены, массивом объектов и извлеченным текстом. После получения ответа выполняется десериализация и переход к этапу нормализации результата.

Нормализация необходима, потому что модель может вернуть дубликаты по регистру, неаккуратные пробелы или несколько записей, описывающих один и тот же объект. Алгоритм приводит названия к нормальной форме, отбрасывает пустые элементы, объединяет дублирующиеся позиции, суммирует количество одинаковых объектов и сортирует итоговый список по имени. Лишь после этого результат выводится в интерфейс и может быть сохранен пользователем в отдельный файл.

Таблица 4.5 - Представление сервиса распознавания объектов как черного ящика

Входные данные	Имя функции	Результат
Массив байтов изображения и MIME-тип	OpenAiObjectRecognitionService.RecognizeAsync	Структурированный объект ImageRecognitionResult с описанием сцены, списком объектов и распознанным текстом

Листинг 4.5 - Псевдокод алгоритма распознавания объектов

1. определить кадр, который требуется анализировать: предпросмотр или текущую рабочую копию;
2. закодировать выбранный кадр в PNG и преобразовать его содержимое в строку Base64;
3. сформировать HTTP-запрос к API мультимодальной модели с инструкцией вернуть JSON по заданной схеме;
4. отправить запрос и дождаться ответа либо сообщения об ошибке;
5. извлечь текст JSON из ответа модели и десериализовать его в объект RecognitionPayload;
6. выполнить нормализацию: обрезать пробелы, исключить пустые элементы, объединить дубли и отсортировать объекты по имени;
7. отобразить итоговый отчет в интерфейсе и предложить пользователю сохранить его в файл TXT или JSON.

4.6 Алгоритмы пользовательского интерфейса и сохранения результатов

Обработчики пользовательских событий играют связующую роль между интерфейсом и вычислительным ядром. Именно они отвечают за открытие системных диалогов, проверку состояния окна, обновление кнопок, изменение текста статуса и вызов методов локальной обработки либо интеллектуального анализа. Несмотря на прикладной характер этих функций, с точки зрения алгоритмизации они важны, так как определяют порядок жизненного цикла данных в приложении.

Сохранение результатов делится на два независимых сценария. В первом сохраняется изображение после локальной обработки, и здесь выбирается графический кодер, соответствующий расширению файла. Во втором сохраняется результат распознавания объектов. При записи текстового отчета

используется уже нормализованный и отсортированный набор данных, что гарантирует единообразный и читаемый итог независимо от того, как именно модель сформулировала исходный ответ.

Таблица 4.6 - Основные обработчики событий пользовательского интерфейса

Событие	Назначение	Ключевой результат
Нажатие кнопки открытия	Выбор файла и загрузка изображения	Создаются исходный и рабочий кадры
Изменение операции или параметра	Пересчет предпросмотра	Пользователь видит новый временный результат
Нажатие кнопки применения	Подтверждение преобразования	Предпросмотр становится рабочей копией
Нажатие кнопки сброса	Возврат к исходному изображению	Восстанавливается начальное состояние кадра
Нажатие кнопки распознавания	Запуск интеллектуального анализа	Формируется отчет о найденных объектах
Нажатие кнопки сохранения	Экспорт изображения или отчета	Создается файл выбранного типа

5 Программное средство

5.1 Состав и организация решения

Программное средство реализовано как решение .NET, включающее несколько проектов с четко распределенной ответственностью. Графический интерфейс располагается в проекте ImageToolkit, а вычислительное ядро сосредоточено в библиотеке ImageToolkit.Core. Подобное разбиение делает архитектуру прозрачной, упрощает сопровождение и позволяет развивать каждую часть системы независимо.

После добавления распознавания объектов в пользовательский проект введен отдельный сервис OpenAiObjectRecognitionService, отвечающий за формирование HTTP-запроса, передачу изображения, разбор структурированного ответа и возврат модели результата в интерфейс. Вспомогательный модуль OpenAiApiKeyResolver инкапсулирует получение ключа из переменной среды либо из локального секрет-файла, что повышает удобство демонстрационного запуска.

Таблица 5.1 - Структура решения ImageToolkit

Проект или модуль	Назначение	Основные обязанности
ImageToolkit	WPF-приложение	Формирование окна, обработка действий пользователя, работа с файлами и вызов сервисов
ImageToolkit.Core	Библиотека обработки данных	Структуры кадра и цвета, каталог операций, алгоритмы локальной обработки, форматирование отчета
OpenAiObjectRecognitionService	Сетевой сервис анализа изображения	Подготовка JSON-запроса, обращение к API, разбор и

		нормализация ответа модели
--	--	-------------------------------

5.2 Описание пользовательского интерфейса

Главное окно программы состоит из панели управления и области просмотра. На панели управления размещены основные действия: открытие изображения, применение выбранной операции, сброс, сохранение результата и запуск распознавания объектов. Ниже располагаются элементы выбора операции и настройки ее параметра, а также блок служебной информации о размере кадра, текущем состоянии и результатах интеллектуального анализа.

Правая часть интерфейса отведена под визуальную работу с данными. В верхней панели размещается единственный крупный просмотрщик изображения, в котором пользователь видит текущее состояние кадра и предпросмотр локального преобразования. Это делает интерфейс менее перегруженным и соответствует сценарию, при котором сравнение двух изображений одновременно не требуется. Ниже располагается отдельный текстовый блок распознавания, где показываются описание сцены, отсортированный список найденных объектов и распознанный текст с изображения. Такое представление делает новую функцию не абстрактной, а практически полезной для пользователя.

5.3 Поддерживаемые функции программы

Таблица 5.2 - Реализованные функции программного средства

Функция	Способ запуска	Результат выполнения
Открытие изображения	Кнопка «Открыть изображение»	Исходный файл загружается и отображается в окне
Предпросмотр операции	Выбор операции и изменение параметра	Временный результат сразу показывается пользователю

Применение преобразования	Кнопка «Применить операцию»	Рабочая копия заменяется новым кадром
Сброс к исходному состоянию	Кнопка «Сбросить к исходному»	Отменяются все накопленные локальные изменения
Сохранение изображения	Кнопка «Сохранить результат»	Создается графический файл PNG, JPEG или BMP
Распознавание объектов	Кнопка «Распознать объекты»	Получается отчет со списком объектов и описанием сцены
Сохранение отчета распознавания	Диалог после распознавания	Создается текстовый файл или JSON-документ

5.4 Порядок работы пользователя с программой

Типовой сценарий начинается с открытия изображения. После выбора файла программа загружает его в память, переводит в единый внутренний формат и отображает в области исходного просмотра. Затем пользователь выбирает одну из доступных операций. Если операция имеет параметр, например смещение яркости или порог бинаризации, пользователь регулирует его ползунком и сразу получает обновленный предпросмотр.

После оценки результата операция может быть применена к рабочей копии изображения. Пользователь вправе повторять эту процедуру несколько раз, комбинируя локальные преобразования в удобной последовательности. Если результат перестал устраивать, всегда можно вернуться к исходному состоянию нажатием кнопки сброса. Финальное изображение сохраняется в графический файл через стандартный диалог Windows.

Для интеллектуального анализа используется отдельная кнопка распознавания объектов. Программа берет текущий кадр, отправляет его в API мультимодальной модели и после получения ответа выводит отсортированный отчет в текстовом поле. Затем пользователю предлагается сохранить отчет в TXT

либо JSON. Такой сценарий особенно удобен в демонстрационных ситуациях, когда необходимо не только визуально обработать изображение, но и быстро получить его содержательное описание.

5.5 Иллюстрации интерфейса программы

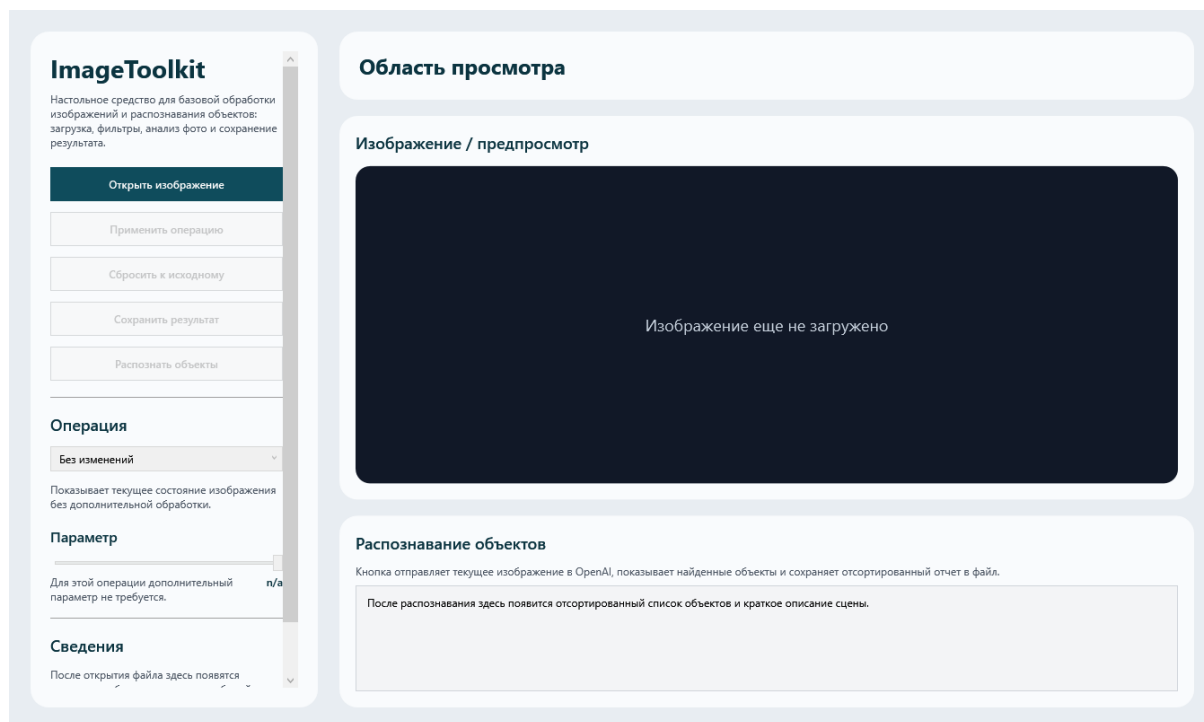


Рисунок 5.1 - Главное окно программы до загрузки изображения

На начальном экране пользователь видит полный набор доступных действий и два поля просмотра. Кнопка распознавания объектов уже присутствует в интерфейсе, однако активируется только после загрузки изображения, что предотвращает неверный сценарий использования и делает поведение программы понятным.

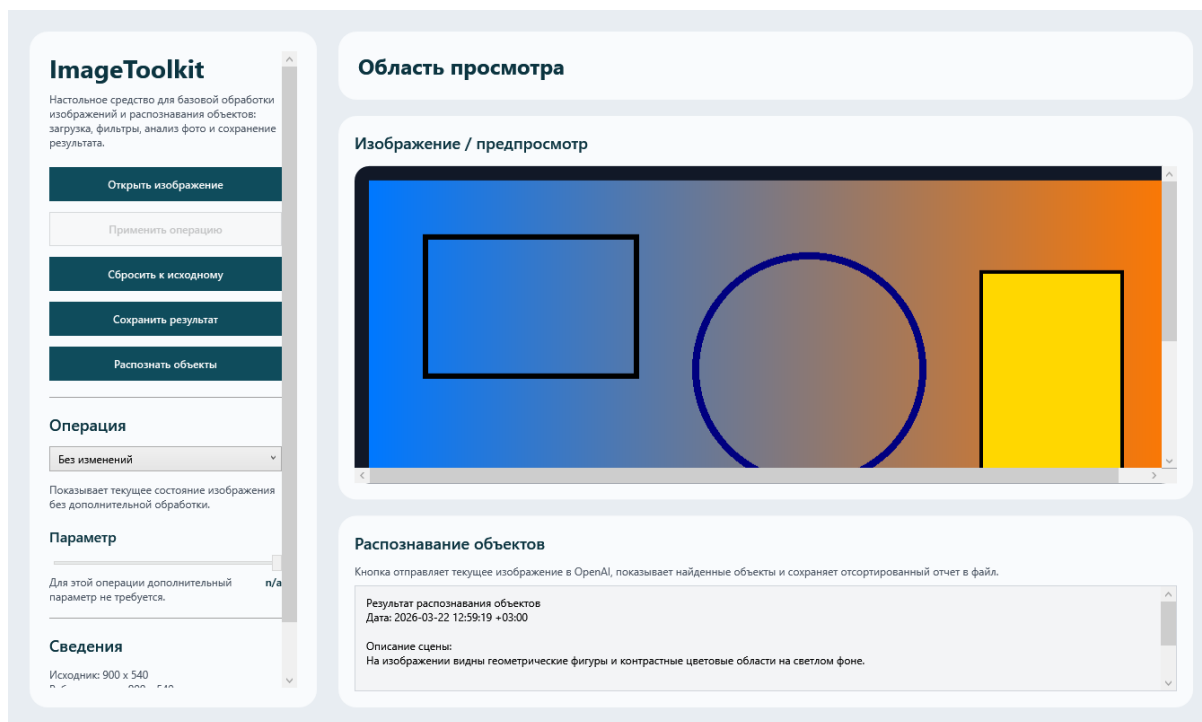


Рисунок 5.2 - Рабочее состояние программы после загрузки изображения и выбора операции

После выбора файла приложение показывает текущее изображение в крупной области просмотра, а ниже выводит блок для результатов распознавания. Пользователь может изменять параметры операции, оценивать предпросмотр, сохранять обработанный кадр и при необходимости переходить к функции распознавания объектов без повторного открытия файла.

5.6 Эксплуатационные особенности и ограничения

Локальные преобразования изображения выполняются полностью на компьютере пользователя и не требуют подключения к сети. Благодаря этому базовый функционал программы сохраняет работоспособность даже в автономном режиме. В то же время функция распознавания объектов зависит от доступности внешнего API, поэтому для нее требуется активное сетевое соединение и корректный ключ доступа.

Для демонстрационного использования предусмотрена возможность чтения ключа как из переменной среды, так и из локального секрет-файла, подключаемого к сборке. Такой подход упрощает перенос программы на другой

компьютер при показе работы, однако в реальной эксплуатации требует внимательного отношения к безопасности ключа и его своевременной замены.

6 Заключение

В ходе выполнения курсовой работы разработано настольное программное средство ImageToolkit, предназначенное для загрузки, визуального анализа и преобразования растровых изображений. В рамках проекта реализована собственная модель хранения кадра, набор локальных алгоритмов обработки, удобный пользовательский интерфейс, подтверждающие корректность ключевых вычислительных операций.

Существенным расширением базовой функциональности стало внедрение интеллектуального распознавания объектов на фотографии. Благодаря интеграции с мультимодальным API программа способна не только изменять вид изображения, но и извлекать из него содержательную информацию. Результат анализа приводится к нормализованному виду, сортируется по имени и сохраняется в отдельный файл, что повышает практическую ценность разработанного решения.

Поставленные в работе задачи выполнены: проведен анализ предметной области, обоснован выбор технологий, разработаны необходимые алгоритмы, реализовано программное средство и подготовлена программная документация. Полученный проект может рассматриваться как завершенный учебный продукт и как основа для дальнейшего развития в сторону более сложных операций обработки изображений и расширенных сценариев интеллектуального анализа визуальных данных.

7 Список использованной литературы

1. ГОСТ 19.101-77 «Виды программ и программных документов».
2. ГОСТ 19.201-78 «Техническое задание. Требования к содержанию и оформлению».
3. ГОСТ 7.32-2017 «Отчет о научно-исследовательской работе. Структура и правила оформления».
4. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework.
5. Троелсен Э., Джепикс Ф. Язык программирования C# и платформа .NET.
6. Официальная документация Microsoft по платформе .NET и языку C#.
7. Официальная документация Microsoft по технологии Windows Presentation Foundation (WPF).
8. Официальная документация Microsoft по классам BitmapSource, BitmapEncoder, OpenFileDialog и SaveFileDialog.
9. Официальная документация OpenAI по работе с мультимодальными моделями, обработке изображений и structured outputs.
10. Исходный код проекта ImageToolkit и библиотека ImageToolkit.Core.
11. Техническое задание на разработку программного средства для обработки изображений ImageToolkit.